

OSSP PRACTICAL WEEK-10

TASK-1:

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

SOURCE CODE:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ mkdir inode_lab
cd inode_lab
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ echo "Hello Inode" > original.txt
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ cat original.txt
Hello Inode
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ ls -li original.txt
43065671436792818 original.txt
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ stat original.txt
  File: original.txt
  Size: 12          Blocks: 0          IO Block: 4096   regular file
Device: 0,70      Inode: 43065671436792818  Links: 1
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/sairam0806)   Gid: ( 1000/sairam0806)
Access: 2026-09-27 14:30:53.124331200 +0000
Modify: 2026-09-27 14:30:52.796058800 +0000
Change: 2026-09-27 14:30:52.796058800 +0000
 Birth: -
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ ln original.txt hardlink.txt
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/inode_lab$ ls -li original.txt hardlink.txt
```

Output:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/inode_lab$ ln original.txt hardlink.txt
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/inode_lab$ ls -li original.txt hardlink.txt
43065671436792818 -rwxrwxrwx 2 sairam0806 sairam0806 12 Sep 27 14:30 hardlink.txt
43065671436792818 -rwxrwxrwx 2 sairam0806 sairam0806 12 Sep 27 14:30 original.txt
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/inode_lab$
```

OBSERVATION:

Hard link shares the same inode as the original file, whereas a symbolic link has a separate inode and stores the path of the original file.

TASK-2:

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

#define FILE_NAME "mmap_test.txt"

double get_time()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1000000000.0;
}

int main()
{
    int fd;
    char buffer[100];
    const char *data = "Hello, this is mmap file I/O.\n";
    size_t size = strlen(data);

    printf("MEMORY MAPPED I/O VS READ/WRITE\n");
    printf("-----\n");

    /* Traditional write() */
    fd = open(FILE_NAME, O_RDWR | O_CREAT | O_TRUNC, 0666);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }
}
```

```
if (fd == -1)
{
    perror("open");
    return 1;
}

double start = get_time();

write(fd, data, size);

double end = get_time();

printf("write() time : %f seconds\n", end - start);

/* Traditional read() */
lseek(fd, 0, SEEK_SET);

start = get_time();

read(fd, buffer, size);

end = get_time();

buffer[size] = '\0';

printf("read() time : %f seconds\n", end - start);
printf("Data: %s", buffer);

/* mmap() */
start = get_time();

char *mapped = mmap(NULL, size,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);

if (mapped == MAP_FAILED)
```

```
buffer[size] = '\\0';
```

```
printf("read() time : %f seconds\\n", end - start);  
printf("Data: %s", buffer);
```

```
/* mmap() */
```

```
start = get_time();
```

```
char *mapped = mmap(NULL, size,  
                    PROT_READ | PROT_WRITE,  
                    MAP_SHARED, fd, 0);
```

```
if (mapped == MAP_FAILED)  
{  
    perror("mmap");  
    close(fd);  
    return 1;  
}
```

```
printf("\\nData using mmap(): %s", mapped);
```

```
strcpy(mapped, "Modified using mmap().\\n");
```

```
msync(mapped, strlen("Modified using mmap().\\n"), MS_SYNC);
```

```
end = get_time();
```

```
printf("mmap() time : %f seconds\\n", end - start);
```

```
munmap(mapped, size);  
close(fd);
```

```
printf("\\nFile operation completed successfully.\\n");
```

```
return 0;
```

```
}
```

Output:

```
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ mkdir mmap_lab  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ cd mmap_lab  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/mmap_lab$ nano mmap_compa  
re.c  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/mmap_lab$ gcc mmap_compar  
e.c -o mmap_compare  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/mmap_lab$ ls  
mmap_compare  mmap_compare.c  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/mmap_lab$ ./mmap_compare  
MEMORY MAPPED I/O VS READ/WRITE  
-----  
write() time : 0.002028 seconds  
read() time  : 0.000583 seconds  
Data: Hello, this is mmap file I/O.  
  
Data using mmap(): Hello, this is mmap file I/O.  
mmap() time   : 0.006243 seconds  
  
File operation completed successfully.  
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806/mmap_lab$ ─
```

OBSERVATION:

The file was successfully mapped into memory using `mmap()`. Data was written directly to the mapped memory and stored in the file.